МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа № 3 по курсу «Операционные системы и архитектура компьютера»

Выполнил: Ф. А. Алексеев
Группа: М8О-204БВ-25
Преподаватель: Н. А. Гапонов
Дата: 05.11.2026
Оценка: _____

Москва, 2026

Условие

Цель работы: Приобретение практических навыков в создании и управлении потоками в ОС, а также их синхронизации.

Задание: Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработки использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска программы.

Вариант 7: Два человека играют в кости. Правила игры следующие: каждый игрок делает бросок 2-ух костей K раз; побеждает тот, кто выбросил суммарно большее количество очков. Задача программы экспериментально определить шансы на победу каждого из игроков. На вход программе подается K, какой сейчас тур, сколько очков суммарно у каждого из игроков и количество экспериментов, которые должна произвести программа.

Метод решения и описание программы

Программа реализует многопоточную архитектуру в рамках одного процесса без создания дочерних процессов на ОС Windows.

Взаимодействие и распределение вычислительной нагрузки организовано по следующему алгоритму:

1. **Разделение вычислений:** Общее число экспериментов равномерно распределяется между заданным количеством потоков. Возможный остаток от деления распределяется поровну между первыми потоками для сохранения точности.
2. **Изоляция потоков:** Каждый поток выполняет симуляции в собственном локальном цикле, накапливая результаты в локальных переменных *(w1, w2, dr)*.
3. **Генерация случайных величин:** Каждый поток инициализирует генератор псевдослучайных чисел собственным уникальным значением *seed*, вычисленным на основе текущего системного времени и номера потока.
4. **Контроль завершения:** Главный поток ожидает окончания работы всех созданных рабочих потоков, после чего производит расчет вероятностей в процентах и выводит результат на экран.

Структура файлов

Таблица 1 – Файлы проекта

Файл	Назначение
`main.c`	Основной процесс – принимает параметры для расчета и вычисляет, создавая заданное число потоков.
`CmakeLists.txt`	Автоматическая сборка проекта.

Основные типы данных и функции

- *ThreadData* (структура): Служит для передачи входных параметров в каждый рабочий поток. Содержит поля *count* (количество симуляций для конкретного потока) и *seed* (базовое число для инициализации генератора случайных чисел).
- *dice_roll*() (функция): Возвращает случайное число от 1 до 6, имитируя бросок одной игральной кости с помощью стандартной функции rand().

- *player_points(long long rounds)* (функция): Симулирует ход одного игрока. Принимает количество бросков и возвращает сумму очков, полученных в результате *rounds* бросков двух костей.
- *worker(LPVOID arg)* (функция): Основная потоковая функция. Принимает указатель на структуру *ThreadData*, инициализирует генератор *srand*(), выполняет цикл симуляций партий и безопасно суммирует результаты в глобальные переменные внутри критической секции.
- *main(int argc, char *argv[])* (функция): Точка входа в программу.

Используемые системные вызовы

Таблица 2 – Используемые системные вызовы

Системный вызов	Назначение в программе
`CreateThread`	Создание и запуск нового вычислительного потока в адресном пространстве текущего процесса с передачей ему функции worker.
`InitializeCriticalSection`	Инициализация объекта критической секции для управления межпоточной синхронизацией.
`WaitForSingleObject`	Блокировка вызова в главном потоке до тех пор, пока указанный рабочий поток (дескриптор *HANDLE*) не завершит свое выполнение.
`GetLastError`	Получение кода последней ошибки от ОС в случае сбоя вызовов *CreateThread* или *WaitForSingleObject*.
`CloseHandle`	Закрытие дескриптора потока и освобождение связанных с ним системных ресурсов после подтверждения его завершения.

Ключевые фрагменты кода

Общее количество симуляций равномерно распределяется между потоками. Остаток от целочисленного деления учитывается при расчете нагрузки для первых потоков. Предусмотрена обработка ошибок: при сбое создания одного из потоков, ранее запущенные потоки продолжают выполнение. Каждому потоку передается уникальное инициализирующее значение (*seed*) для обеспечения независимости генерации случайных чисел.

Листинг 1 – Инициализация всех потоков через цикл

```
…
    long long base = n / threads;
    long long extra = n % threads;
    for (int i = 0; i < threads; i++) {
        data[i].count = base + (i < extra ? 1 : 0);
        data[i].seed = (unsigned int)time(NULL) ^ (i * 16777619U);

        handles[i] = CreateThread(NULL, 0, worker, &data[i], 0, NULL);
        if (handles[i] == NULL) {
            fprintf(stderr, "CreateThread failed on %d: error %lu\n", i+1,
GetLastError());
            fprintf(stdout, "Were able to create only %d threads\n", i);
            error = 1;
            break;
        }
```

```
        created++;
    }
```

Потоковая функция *worker* принимает указатель на структуру данных, выполняет локальный цикл симуляций и обновляет промежуточные результаты в локальных переменных *w1, w2, dr*. Запись результатов в глобальные переменные защищена критической секцией для предотвращения состояния *Race condition.*

Листинг 2 – Функция вычисления внутри каждого потока

```
DWORD WINAPI worker(LPVOID arg) {
    ThreadData *data = (ThreadData *)arg;
    long long w1 = 0, w2 = 0, dr = 0;
    srand(data->seed);
    for (long long i = 0; i < data->count; i++) {
        long long s1 = g_p1 + player_points(g_K);
        long long s2 = g_p2 + player_points(g_K);
        if (s1 > s2) w1++;
        else if (s2 > s1) w2++;
        else dr++;
    }
    EnterCriticalSection(&g_lock);
    g_wins1 += w1;
    g_wins2 += w2;
    g_draws += dr;
    LeaveCriticalSection(&g_lock);
    return 0;
}
```

Результаты

Программа производит случайные генерации и на их основе вычисляет вероятность выигрыша первого игрока, второго, а также шанс ничьи. Примеры входных данных и результаты работы (вывод) программы представлены ниже:

Листинг 1 – Обычный пример

```
$ build/main
Enter K, p1 score, p2 score, n: 500 100 150 50000
Player 1 wins: 25.402000%
Player 2 wins: 74.146000%
Draws: 0.452000%
```

Листинг 2 – Повторный запуск с входными данными из Листинга 1

```
$ build/main
Enter K, p1 score, p2 score, n: 500 100 150 50000
Player 1 wins: 25.418000%
Player 2 wins: 74.148000%
Draws: 0.434000%
```

Листинг 3 – Гарантированная победа одной из сторон

```
$ build/main
Enter K, p1 score, p2 score, n: 100 500 100 50000
Player 1 wins: 100.000000%
Player 2 wins: 0.000000%
Draws: 0.000000%
```

Зависимость времени выполнения программы от числа потоков измерялась на неизменных входных данных: *K = 1000, p1 = 500, p2 = 500, n = 50000.* Результаты измерений зависимости приведены ниже:

Таблица 3 – Зависимость времени выполнения от числа потоков

Число потоков	Время, мс	Ускорение	Эффективность
1	1302,316	1,000	1,000
2	693,236	1,879	0,939
4	350,355	3,717	0,929
8	221,721	5,874	0,734
16	127,277	10,232	0,640

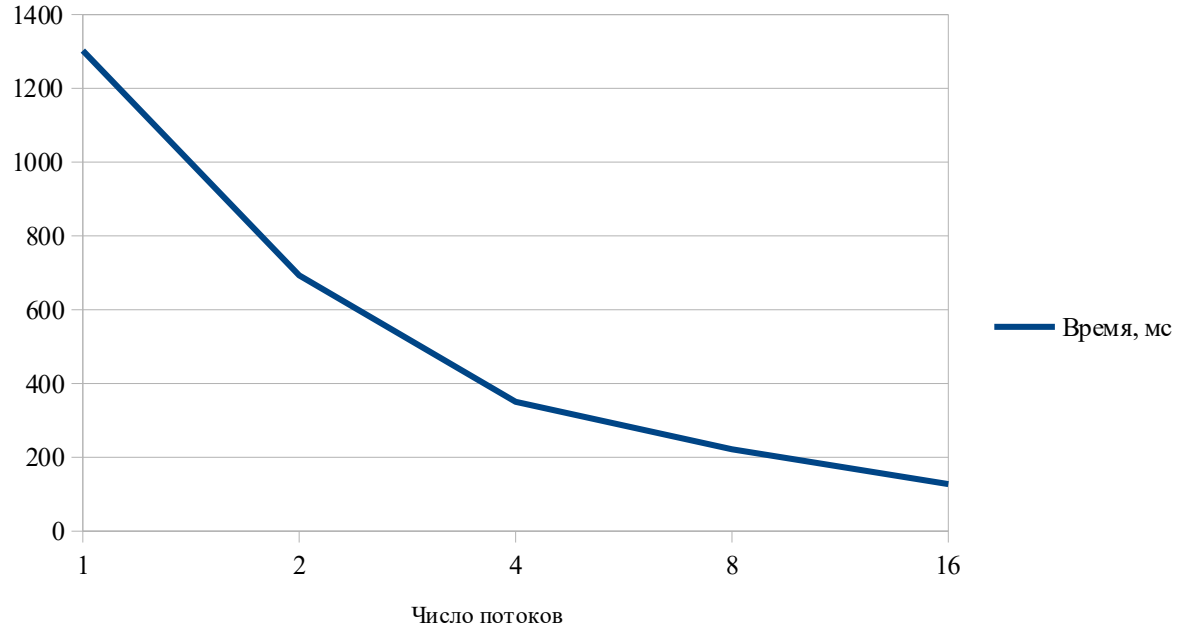


Рисунок 1 – Зависимость времени выполнения от числа потоков

Демонстрация работы Process Monitor

Программа запущена с 4 потоками; полный вывод приведён в приложении Б.

```
16:04:53,2790130    main.exe      26936  Process Start        SUCCESS Parent PID:
21112, Command line: "C:\Users\fedor\OneDrive\Документы\os_lab3\src\build\main.exe" 4,
Current directory: C:\Users\fedor\OneDrive\Документы\os_lab3\src\
16:04:53,2790176    main.exe      26936  Thread Create        SUCCESS Thread ID: 26828
```

Инициализируется процесс *main.exe* и создается его главный поток.

```
16:04:53,2860056    main.exe      26936  Thread Create        SUCCESS Thread ID: 27488
16:04:53,2860210    main.exe      26936  Thread Create        SUCCESS Thread ID: 27444
16:04:53,2860338    main.exe      26936  Thread Create        SUCCESS Thread ID: 21500
16:04:53,2860458    main.exe      26936  Thread Create        SUCCESS Thread ID: 24364
16:04:53,6200254    main.exe      26936  Thread Exit          SUCCESS Thread ID: 27444,
User Time: 0.1250000, Kernel Time: 0.2031250
16:04:53,6246447    main.exe      26936  Thread Exit          SUCCESS Thread ID: 21500,
User Time: 0.1406250, Kernel Time: 0.2031250
```

```
16:04:53,6297666    main.exe       26936  Thread Exit          SUCCESS Thread ID: 27488,
User Time: 0.2812500, Kernel Time: 0.0156250
16:04:53,6318934    main.exe       26936  Thread Exit          SUCCESS Thread ID: 24364,
User Time: 0.2656250, Kernel Time: 0.0625000
```

Главный поток порождает 4 дополнительных вычислительных потока. Ниже зафиксировано успешное завершение работы каждого из них (*Thread Exit*).

Выводы

В ходе выполнения лабораторной работы была успешно реализована многопоточная программа на языке Си для ОС Windows, моделирующая игру подбрасывания костей с помощью системного вызова *CreateThread*. Корректность работы подтверждается результатами тестирования, стабильным ускорением времени выполнения при увеличении числа потоков, а также логичным поведением метрик эффективности параллелизма. Основная трудность заключалась в генерации псевдослучайных чисел, так как при одновременном запуске потоков стандартный метод инициализации по времени *(time(NULL))* приводил к совпадению начальных значений сида и, как следствие, к идентичным результатам вычислений. Данная проблема была решена путем модификации *seed* для каждого отдельного потока с помощью операции между текущим системным временем и специальной константой, умноженной на индекс потока, что обеспечило независимость выборок.

Приложение А. Исходный код программы

Листинг А.1 – Файл main.c

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

typedef struct {
    long long count;
    unsigned int seed;
} ThreadData;

static long long g_K, g_p1, g_p2;
static long long g_wins1 = 0, g_wins2 = 0, g_draws = 0;
static CRITICAL_SECTION g_lock, g_rand_lock;

int dice_roll() {
    return 1 + rand() % 6;
}

long long player_points(long long rounds) {
    long long sum = 0;
    for (long long i = 0; i < rounds; i++) {
        sum += dice_roll();
        sum += dice_roll();
    }
    return sum;
}

DWORD WINAPI worker(LPVOID arg) {
    ThreadData *data = (ThreadData *)arg;
    long long w1 = 0, w2 = 0, dr = 0;
    srand(data->seed);
    for (long long i = 0; i < data->count; i++) {
        long long s1 = g_p1 + player_points(g_K);
        long long s2 = g_p2 + player_points(g_K);
        if (s1 > s2) w1++;
        else if (s2 > s1) w2++;
        else dr++;
    }
    EnterCriticalSection(&g_lock);
    g_wins1 += w1;
    g_wins2 += w2;
    g_draws += dr;
    LeaveCriticalSection(&g_lock);
    return 0;
}

int main(int argc, char *argv[]) {
    int threads;
    if (argc < 2) threads = 1;
    else {
```

```
        char *end;
        threads = strtol(argv[1], &end, 10);
        if (end == argv[1] || *end != '\0' || threads < 1) {
            fprintf(stderr, "Threads count must be a positive int: %d", threads);
            return 1;
        }
    }
    long long n;
    printf("Enter K, p1 score, p2 score, n: ");
    if (scanf("%lld %lld %lld %lld", &g_K, &g_p1, &g_p2, &n) != 4) {
        fprintf(stderr, "Error: expected 4 integers\n");
        return 1;
    }
    if (g_K < 1 || g_p1 < 0 || g_p2 < 0 || n < 1) {
        fprintf(stderr, "Error: need K >= 1, p1 >= 0, p2 >= 0, n >= 1\n");
        return 1;
    }

    HANDLE handles[threads];
    ThreadData data[threads];
    int created = 0;
    int error_code = 0;

    InitializeCriticalSection(&g_lock);

    long long base = n / threads;
    long long extra = n % threads;

    for (int i = 0; i < threads; i++) {
        data[i].count = base + (i < extra ? 1 : 0);
        data[i].seed = (unsigned int)time(NULL) ^ (i * 16777619U);

        handles[i] = CreateThread(NULL, 0, worker, &data[i], 0, NULL);
        if (handles[i] == NULL) {
            fprintf(stderr, "CreateThread failed on %d: error %lu\n", i+1,
GetLastError());
            fprintf(stdout, "Were able to create only %d threads\n", i);
            error_code = 1;
            break;
        }
        created++;
    }

    for (int i = 0; i < created; i++) {
        if (WaitForSingleObject(handles[i], INFINITE) == WAIT_FAILED) {
            fprintf(stderr, "WaitForSingleObject failed: error %lu\n", GetLastError());
            error_code = 1;
        }
        CloseHandle(handles[i]);
    }
    DeleteCriticalSection(&g_lock);
```

```
    printf("Player 1 wins: %f%%\n", 100.0 * g_wins1 / n);
    printf("Player 2 wins: %f%%\n", 100.0 * g_wins2 / n);
    printf("Draws: %f%%\n", 100.0 * g_draws / n);
    return error_code;
}
```

Листинг А.2 – Файл CMakeLists.txt

```
cmake_minimum_required(VERSION 4.2.3)
project(lab3 LANGUAGES C)
set(CMAKE_C_STANDARD 11)

add_executable(main main.c)
```

Приложение Б. Полный вывод Process Monitor

Листинг Б.1 – Вывод Process Monitor

```
16:04:53,2790130    main.exe       26936   Process Start          SUCCESS Parent PID:
21112, Command line: "C:\Users\fedor\OneDrive\Документы\os_lab3\src\build\main.exe" 4,
Current directory: C:\Users\fedor\OneDrive\Документы\os_lab3\src\
16:04:53,2790176    main.exe       26936   Thread Create          SUCCESS Thread ID: 26828
16:04:53,2815670    main.exe       26936   Load Image     C:\Users\fedor\OneDrive\
Документы\os_lab3\src\build\main.exe      SUCCESS Image Base: 0x7ff762b90000, Image Size:
0x15000
16:04:53,2815980    main.exe       26936   Load Image     C:\Windows\System32\ntdll.dll
      SUCCESS Image Base: 0x7ffbc3400000, Image Size: 0x268000
16:04:53,2816664    main.exe       26936   CreateFile     C:\Windows\Prefetch\MAIN.EXE-
774114D6.pf  SUCCESS Desired Access: Generic Read, Disposition: Open, Options:
Synchronous IO Non-Alert, Attributes: n/a, ShareMode: None, AllocationSize: n/a,
OpenResult: Opened
16:04:53,2816902    main.exe       26936   QueryEAFile    C:\Windows\Prefetch\MAIN.EXE-
774114D6.pf  SUCCESS
16:04:53,2822938    main.exe       26936   QueryStandardInformationFile C:\Windows\
Prefetch\MAIN.EXE-774114D6.pf      SUCCESS AllocationSize: 4 096, EndOfFile: 1 978,
NumberOfLinks: 1, DeletePending: False, Directory: False
16:04:53,2823121    main.exe       26936   ReadFile       C:\Windows\Prefetch\MAIN.EXE-
774114D6.pf  SUCCESS Offset: 0, Length: 1 978, Priority: Normal
16:04:53,2823994    main.exe       26936   CloseFile      C:\Windows\Prefetch\MAIN.EXE-
774114D6.pf  SUCCESS
16:04:53,2843287    main.exe       26936   CreateFile     C:\Users\fedor\OneDrive\
Документы\os_lab3\src       SUCCESS Desired Access: Execute/Traverse, Synchronize,
Disposition: Open, Options: Directory, Synchronous IO Non-Alert, Attributes: n/a,
ShareMode: Read, Write, AllocationSize: n/a, OpenResult: Opened
16:04:53,2844530    main.exe       26936   Load Image     C:\Windows\System32\kernel32.dll
      SUCCESS Image Base: 0x7ffbc1f20000, Image Size: 0xcb000
16:04:53,2845291    main.exe       26936   Load Image     C:\Windows\System32\
KernelBase.dll      SUCCESS Image Base: 0x7ffbc0780000, Image Size: 0x401000
16:04:53,2855223    main.exe       26936   Load Image     C:\Windows\System32\ucrtbase.dll
      SUCCESS Image Base: 0x7ffbc0dc0000, Image Size: 0x14c000
16:04:53,2855938    main.exe       26936   Thread Create          SUCCESS Thread ID: 17684
16:04:53,2859215    main.exe       26936   QueryNameInformationFile     C:\Users\fedor\
OneDrive\Документы\os_lab3\src\build\main.exe     SUCCESS Name: \Users\fedor\OneDrive\
Документы\os_lab3\src\build\main.exe
16:04:53,2860056    main.exe       26936   Thread Create          SUCCESS Thread ID: 27488
16:04:53,2860210    main.exe       26936   Thread Create          SUCCESS Thread ID: 27444
16:04:53,2860338    main.exe       26936   Thread Create          SUCCESS Thread ID: 21500
16:04:53,2860458    main.exe       26936   Thread Create          SUCCESS Thread ID: 24364
16:04:53,6200254    main.exe       26936   Thread Exit            SUCCESS Thread ID: 27444,
User Time: 0.1250000, Kernel Time: 0.2031250
16:04:53,6246447    main.exe       26936   Thread Exit            SUCCESS Thread ID: 21500,
User Time: 0.1406250, Kernel Time: 0.2031250
16:04:53,6297666    main.exe       26936   Thread Exit            SUCCESS Thread ID: 27488,
User Time: 0.2812500, Kernel Time: 0.0156250
16:04:53,6318934    main.exe       26936   Thread Exit            SUCCESS Thread ID: 24364,
User Time: 0.2656250, Kernel Time: 0.0625000
16:04:53,6321962    main.exe       26936   CreateFile     C:\Windows\System32\
kernel.appcore.dll  SUCCESS Desired Access: Read Attributes, Disposition: Open, Options:
```

```
Open Reparse Point, Attributes: n/a, ShareMode: Read, Write, Delete, AllocationSize:
n/a, OpenResult: Opened
16:04:53,6322157    main.exe       26936  QueryBasicInformationFile    C:\Windows\
System32\kernel.appcore.dll SUCCESS CreationTime: Ср, 09 сен 2026 8:59:17, LastAccessTime:
Вс, 01 ноя 2026 16:04:53, LastWriteTime: Ср, 09 сен 2026 8:59:17, ChangeTime: Чт, 15 окт
2026 12:26:33, FileAttributes: A
16:04:53,6322213    main.exe       26936  CloseFile      C:\Windows\System32\
kernel.appcore.dll  SUCCESS
16:04:53,6322836    main.exe       26936  CreateFile     C:\Windows\System32\
kernel.appcore.dll  SUCCESS Desired Access: Read Data/List Directory, Execute/Traverse,
Synchronize, Disposition: Open, Options: Synchronous IO Non-Alert, Non-Directory File,
Attributes: n/a, ShareMode: Read, Delete, AllocationSize: n/a, OpenResult: Opened
16:04:53,6323027    main.exe       26936  CreateFileMapping     C:\Windows\System32\
kernel.appcore.dll  FILE LOCKED WITH ONLY READERS SyncType: SyncTypeCreateSection,
PageProtection: PAGE_EXECUTE
16:04:53,6323146    main.exe       26936  CreateFileMapping     C:\Windows\System32\
kernel.appcore.dll  SUCCESS SyncType: SyncTypeOther
16:04:53,6324119    main.exe       26936  Load Image     C:\Windows\System32\
kernel.appcore.dll  SUCCESS Image Base: 0x7ffbbf250000, Image Size: 0x1b000
16:04:53,6325356    main.exe       26936  Load Image     C:\Windows\System32\msvcrt.dll
     SUCCESS Image Base: 0x7ffbc1110000, Image Size: 0xa9000
16:04:53,6326744    main.exe       26936  CloseFile      C:\Windows\System32\
kernel.appcore.dll  SUCCESS
16:04:53,6328234    main.exe       26936  Thread Exit           SUCCESS Thread ID: 17684,
User Time: 0.0000000, Kernel Time: 0.0000000
16:04:53,6328757    main.exe       26936  Thread Exit           SUCCESS Thread ID: 26828,
User Time: 0.0000000, Kernel Time: 0.0000000
16:04:53,6330833    main.exe       26936  Process Exit          SUCCESS Exit Status: 0,
User Time: 0.8281250 seconds, Kernel Time: 0.4843750 seconds, Private Bytes: 663 552,
Peak Private Bytes: 704 512, Working Set: 5 025 792, Peak Working Set: 5 025 792
16:04:53,6331911    main.exe       26936  CloseFile      C:\Users\fedor\OneDrive\
Документы\os_lab3\src       SUCCESS
```